Algoritmos de clasificación

M6: Aprendizaje de máquina supervisado

|**AE5**: Elaborar un modelo predictivo utilizando algoritmos de clasificación para resolver un problema de baja complejidad acorde a las buenas prácticas de la industria.

alkemy

Introducción👩‍💻

Los algoritmos de clasificación son herramientas fundamentales en el campo del aprendizaje automático supervisado. Su principal objetivo es asignar una etiqueta o categoría a una observación desconocida, basándose en patrones aprendidos a partir de un conjunto de datos etiquetados. Entre los algoritmos más sencillos y efectivos para este propósito se encuentra el algoritmo **K-Nearest Neighbors (K-NN)**.

K-NN es un algoritmo basado en la proximidad y la similitud entre observaciones. Su principio es sencillo: para clasificar un nuevo dato, el algoritmo busca los *k* datos más cercanos a él en el conjunto de entrenamiento y asigna la clase más común entre ellos. Este enfoque hace que K-NN sea un algoritmo intuitivo, fácil de implementar y eficaz en múltiples escenarios.

A lo largo de este manual, exploraremos el funcionamiento detallado del algoritmo K-NN, así como conceptos clave relacionados, como el escalamiento de datos y la distancia euclideana. También analizaremos la importancia de seleccionar un valor adecuado para *k* y evaluaremos las ventajas y desventajas de utilizar este algoritmo. Finalmente, implementaremos un modelo K-NN utilizando la librería **Scikit-Learn**, aplicando buenas prácticas para su entrenamiento y evaluación.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Comprender en qué consiste el algoritmo K-Nearest Neighbors.
- Explicar la importancia del escalamiento de datos para el uso de K-NN.
- Interpretar el concepto de distancia euclideana y su rol en K-NN.
- Determinar la manera adecuada de seleccionar el valor de *k* y evaluar la exactitud del modelo.
- Identificar las ventajas y desventajas de utilizar el algoritmo K-NN.
- Implementar un clasificador K-NN utilizando la librería **Scikit-Learn** en Python.
- Evaluar el desempeño del modelo mediante métricas adecuadas.

Algoritmos de Clasificación

1. En qué consiste el algoritmo K-Nearest Neighbors (K-NN)

El algoritmo **K-Nearest Neighbors (K-NN)** es un método de clasificación supervisada que asigna una categoría a una nueva observación basándose en las clases de sus vecinos más cercanos. Es un algoritmo de tipo **lazy learning** (aprendizaje perezoso), ya que no construye un modelo explícito durante el entrenamiento, sino que almacena los datos y realiza los cálculos en el momento de la predicción.

El funcionamiento de K-NN es sencillo e intuitivo. Ante una nueva observación, el algoritmo calcula la distancia entre esta observación y todos los datos del conjunto de entrenamiento. A continuación, identifica los **k** elementos más cercanos y asigna la clase más común entre ellos. La elección de *k* es un hiperparámetro clave que puede afectar significativamente el rendimiento del modelo.

El algoritmo K-NN se basa en la premisa de que las observaciones cercanas entre sí tienden a pertenecer a la misma clase. Por lo tanto, la precisión del modelo dependerá de la distribución y calidad de los datos de entrenamiento, así como de la elección adecuada de la métrica de distancia y el valor de *k*.

K-NN es un algoritmo **no paramétrico**, lo que significa que no hace suposiciones explícitas sobre la distribución de los datos. Esto lo convierte en una opción flexible para problemas en los que no se conoce la relación entre las variables independientes y la variable objetivo.

El entrenamiento de K-NN es muy rápido, ya que consiste simplemente en almacenar el conjunto de datos. Sin embargo, el costo computacional de la predicción puede ser elevado, especialmente para conjuntos de datos grandes, ya que requiere calcular la distancia entre la nueva observación y todos los datos existentes.

K-NN es sensible a la escala de las variables, por lo que es necesario aplicar técnicas de **escalamiento de datos** antes de utilizar este algoritmo. De lo contrario, las variables con mayor magnitud dominarán el cálculo de la distancia y afectarán negativamente la clasificación.

Además, K-NN puede verse afectado por la presencia de variables irrelevantes o ruidosas, ya que estas pueden distorsionar la medición de distancia y reducir la precisión del modelo. Por ello, es recomendable realizar un análisis exploratorio y una selección adecuada de características antes de aplicar K-NN.

Ejemplo práctico

Supongamos que tenemos el siguiente conjunto de datos:

ID	Altura (cm)	Peso (kg)	Deporte favorito
1	180	75	Fútbol
2	165	65	Baloncesto
3	170	70	Fútbol
4	160	60	Baloncesto

Queremos predecir el deporte favorito de una persona con altura 172 cm y peso 72 kg. K-NN calculará la distancia de esta nueva observación con las existentes, buscará los k vecinos más cercanos (por ejemplo, $k = 3$) y asignará la clase más común. En este caso, es probable que la predicción sea **Fútbol**, ya que los vecinos más cercanos tienen esa categoría.

2. El concepto de escalamiento de datos

El **escalamiento de datos** es una técnica fundamental en el preprocesamiento de información para algoritmos de aprendizaje automático, especialmente en aquellos que dependen de la medición de distancias, como K-Nearest Neighbors (K-NN). El objetivo del escalamiento es normalizar las variables numéricas para que todas tengan un rango similar y ninguna domine el cálculo de la distancia.

Cuando las variables de un conjunto de datos tienen escalas muy diferentes, el cálculo de distancias puede verse distorsionado. Por ejemplo, si una variable mide altura en centímetros (con valores entre 150 y 200) y otra mide ingresos

anuales (con valores entre 10,000 y 100,000), la segunda variable tendrá un impacto mucho mayor en la distancia, incluso si es menos relevante para la clasificación.

Existen varias técnicas de escalamiento de datos. Una de las más utilizadas es la **normalización Min-Max**, que transforma los valores de cada variable para que estén en un rango específico, generalmente entre 0 y 1. La fórmula aplicada es:

$$X_{\text{escalado}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Otra técnica común es la **estandarización (Z-Score Scaling)**, que transforma los datos para que tengan una media de 0 y una desviación estándar de 1. La fórmula es:

$$X_{\text{escalado}} = \frac{X - \mu}{\sigma}$$

donde μ\muμ es la media y σ\sigmaσ la desviación estándar de la variable.

La elección entre normalización y estandarización depende del algoritmo y la naturaleza de los datos. En el caso de K-NN, ambas técnicas son válidas, ya que garantizan que todas las variables contribuyan equitativamente al cálculo de la distancia.

Es importante aplicar el escalamiento después de dividir los datos en conjunto de entrenamiento y prueba para evitar **data leakage**. Además, el escalador debe ser ajustado únicamente con los datos de entrenamiento y luego aplicado tanto a los datos de entrenamiento como a los de prueba.

El escalamiento también facilita la convergencia de algunos algoritmos y mejora la visualización de los datos en gráficos y análisis exploratorios. En K-NN, es un paso esencial que no puede omitirse.

Ejemplo práctico y resolución

Supongamos que tenemos los siguientes datos:

Altura (cm)	Peso (kg)

180	75

165	65

170	70

160	60

Aplicamos Min-Max Scaling y estandarización utilizando Scikit-Learn:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Datos
datos = pd.DataFrame({
    'Altura': [180, 165, 170, 160],
    'Peso': [75, 65, 70, 60]
})
```

```
# Min-Max Scaling
minmax_scaler = MinMaxScaler()
datos_minmax = minmax_scaler.fit_transform(datos)

print("Min-Max Scaling:")
print(datos_minmax)

# Estandarización
standard_scaler = StandardScaler()
datos_standard = standard_scaler.fit_transform(datos)

print("\nEstandarización:")
print(datos_standard)
```

Salida:

```
Min-Max Scaling:
[[1.   1.  ]
 [0.33333333 0.33333333]
 [0.66666667 0.66666667]
 [0.        0.        ]]

Estandarización:
[[ 1.34164079  1.34164079]
 [-0.4472136  -0.4472136 ]
 [ 0.4472136   0.4472136 ]
 [-1.34164079 -1.34164079]]
```

3. El concepto de distancia euclideana

La **distancia euclideana** es una de las métricas más utilizadas para calcular la separación entre dos puntos en un espacio n-dimensional. Su simplicidad y significado geométrico la convierten en la opción predeterminada en muchos algoritmos de aprendizaje automático basados en proximidad, como el algoritmo K-Nearest Neighbors (K-NN).

Esta distancia mide la longitud del segmento de línea recta que conecta dos puntos. En un espacio bidimensional, la fórmula de la distancia euclideana entre dos puntos A=(x1,y1)A = (x_1, y_1)A=(x1,y1) y B=(x2,y2)B = (x_2, y_2)B=(x2,y2) es:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

En espacios de más dimensiones, la fórmula se extiende sumando las diferencias al cuadrado de todas las variables. La distancia euclideana tiene un significado geométrico intuitivo y es invariante ante traslaciones y rotaciones del espacio.

En el algoritmo K-NN, la distancia euclideana se utiliza para determinar cuáles son los vecinos más cercanos a una nueva observación. El algoritmo calcula esta distancia entre la observación desconocida y todos los datos del conjunto de entrenamiento.

Sin embargo, la distancia euclideana puede verse afectada por la escala de las variables. Si una variable tiene un rango mucho mayor que las demás, dominará

el cálculo de la distancia. Por ello, es imprescindible aplicar técnicas de escalamiento antes de utilizar K-NN con distancia euclideana.

Otra limitación de esta métrica es que no es adecuada para variables categóricas, ya que solo puede calcularse con variables numéricas. Por lo tanto, antes de aplicar K-NN, las variables categóricas deben transformarse mediante técnicas como One-Hot Encoding o variables dummy.

A pesar de sus limitaciones, la distancia euclideana es muy eficaz cuando los datos han sido correctamente preprocesados. Además, existen otras métricas de distancia, como la distancia Manhattan o la distancia de Minkowski, que pueden ser consideradas según la naturaleza de los datos.

Ejemplo práctico y resolución

Supongamos que tenemos los siguientes dos puntos:

- A = (180, 75)
- B = (170, 70)

La distancia euclideana entre ambos es:

$$d(A,B) = \sqrt{(170-180)^2 + (70-75)^2}$$

$$d(A,B) = \sqrt{(-10)^2 + (-5)^2} = \sqrt{100+25} = \sqrt{125} \approx 11.18$$

En Python, podemos calcularlo fácilmente:

```python
from scipy.spatial import distance

punto_a = [180, 75]
punto_b = [170, 70]

distancia = distance.euclidean(punto_a, punto_b)
print(f"Distancia euclideana: {distancia:.2f}")
```

Salida:

```
Distancia euclideana: 11.18
```

Este valor indica qué tan cercanos están ambos puntos. El algoritmo K-NN utilizará este cálculo para decidir qué clase asignar a una nueva observación, basándose en los vecinos con distancia mínima.

4. Selección del valor de K y la exactitud

Uno de los elementos clave en el algoritmo K-Nearest Neighbors (K-NN) es la correcta selección del valor de **K**, es decir, la cantidad de vecinos que se tomarán en cuenta para determinar la clase de una nueva observación. La elección de este parámetro tiene un impacto directo en la precisión y la capacidad de generalización del modelo.

Cuando el valor de K es muy pequeño, por ejemplo, K = 1, el modelo puede ser muy sensible a los datos de entrenamiento, incluyendo posibles errores o ruido. Esto genera un problema conocido como **sobreajuste (overfitting)**, donde el modelo se ajusta demasiado a los datos de entrenamiento, pero su desempeño en datos nuevos es deficiente.

Por otro lado, si K es demasiado grande, el modelo pierde sensibilidad y puede cometer errores al incluir vecinos que no son relevantes para la clasificación. Este fenómeno se conoce como **subajuste (underfitting)**, ya que el modelo es demasiado simple y no captura adecuadamente la estructura de los datos.

No existe un valor de K óptimo universal. La elección adecuada depende de la naturaleza y tamaño del conjunto de datos. Un enfoque común es probar diferentes valores de K y seleccionar aquel que minimice el error en un conjunto de validación o mediante técnicas de validación cruzada.

Una estrategia práctica para determinar el valor de K es evaluar el **porcentaje de exactitud (accuracy)** para cada valor posible de K y graficar el resultado. Esto permite identificar el punto donde la exactitud es máxima y evitar tanto el sobreajuste como el subajuste.

El cálculo de la exactitud consiste en comparar las predicciones del modelo con las etiquetas reales del conjunto de prueba y calcular el porcentaje de aciertos. Esta métrica es adecuada cuando las clases están balanceadas.

Además de la exactitud, es recomendable analizar otras métricas de evaluación, como la matriz de confusión, la precisión, la sensibilidad y la puntuación F1, especialmente en problemas de clasificación con clases desbalanceadas.

Ejemplo práctico y resolución

Utilizando el conjunto de datos de altura y peso, podemos probar diferentes valores de K y observar el efecto en la exactitud:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Datos
datos = pd.DataFrame({
    'Altura': [180, 165, 170, 160],
    'Peso': [75, 65, 70, 60],
    'Deporte': ['Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto']
})
```

```
X = datos[['Altura', 'Peso']]
y = datos['Deporte']

# Escalamiento
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# División de datos
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.5, random_state=
```

```python
exactitudes = []
k_values = range(1, 4)

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    exactitud = accuracy_score(y_test, y_pred)
    exactitudes.append(exactitud)

# Mostrar resultados
plt.plot(k_values, exactitudes, marker='o')
plt.xlabel('Valor de K')
plt.ylabel('Exactitud')
plt.title('Selección del valor de K')
plt.grid(True)
plt.show()
```

Este gráfico permitirá observar cómo varía la exactitud según el valor de K y ayudará a elegir el más adecuado para el conjunto de datos.

5. Ventajas y desventajas de utilizar el algoritmo

El algoritmo **K-Nearest Neighbors (K-NN)** es uno de los más sencillos y utilizados en tareas de clasificación supervisada. Su simplicidad y naturaleza intuitiva hacen que sea una opción atractiva para problemas en los que se requiere rapidez y facilidad de implementación.

Entre sus principales **ventajas**, destaca que no requiere un proceso complejo de entrenamiento. El algoritmo simplemente almacena los datos y realiza los cálculos en el momento de la predicción, lo que lo convierte en un método de **aprendizaje perezoso (lazy learning)**. Esto facilita la implementación y ahorra tiempo en la fase inicial.

Otra ventaja importante es que K-NN es un algoritmo **no paramétrico**, lo que significa que no hace suposiciones explícitas sobre la distribución de los datos. Esto le otorga una gran flexibilidad para trabajar con diferentes tipos de datos y relaciones no lineales.

Además, K-NN puede adaptarse fácilmente a problemas multicategoría y no requiere ajustes significativos para manejar múltiples clases. También permite incorporar nuevas observaciones al conjunto de datos de manera sencilla, sin necesidad de reentrenar el modelo.

Sin embargo, K-NN también presenta varias **desventajas** que deben tenerse en cuenta. Una de las principales es el **alto coste computacional** en la fase de predicción. Dado que el modelo necesita calcular la distancia entre la nueva observación y todos los datos de entrenamiento, el tiempo de predicción aumenta considerablemente cuando el conjunto de datos es grande.

Otra desventaja es que el rendimiento de K-NN es muy sensible a la **escala de las variables**. Si las variables no han sido previamente normalizadas o estandarizadas, aquellas con mayores rangos tendrán un impacto desproporcionado en el cálculo de la distancia.

El algoritmo también es vulnerable a la presencia de **ruido y variables irrelevantes**, ya que todas las variables contribuyen al cálculo de la distancia, independientemente de su relevancia para la tarea de clasificación.

Finalmente, la elección del valor de **K** puede ser un desafío. Un valor muy pequeño puede llevar al modelo a sobreajustarse a los datos de entrenamiento, mientras que un valor muy grande puede hacer que el modelo sea insensible a patrones locales.

Resumen práctico

A continuación, se presenta un resumen de las ventajas y desventajas de K-NN:

Ventajas	Desventajas
Fácil de implementar e interpretar	Coste computacional alto en predicción
No requiere entrenamiento previo	Sensible a la escala de las variables
No paramétrico (no hace suposiciones sobre los datos)	Afectado por ruido y variables irrelevantes

Admite múltiples clases	**Requiere selección cuidadosa del valor de K**

Flexibilidad para incorporar nuevos datos	**No apto para conjuntos de datos muy grandes**

El conocimiento de estas ventajas y desventajas es fundamental para decidir cuándo y cómo aplicar el algoritmo K-NN en un proyecto de aprendizaje automático.

6. Implementación en Python con la librería Scikit-Learn

La librería **Scikit-Learn** es una de las más utilizadas para implementar modelos de aprendizaje automático en Python. Esta herramienta ofrece un módulo específico para el algoritmo **K-Nearest Neighbors (K-NN)**, facilitando todo el proceso de clasificación, desde el preprocesamiento de los datos hasta la evaluación del modelo.

El proceso de implementación comienza con la preparación de los datos. Es importante asegurarse de que los datos estén correctamente estructurados, sin valores faltantes y con las variables categóricas codificadas de manera adecuada. Además, se debe aplicar un **escalamiento** a las variables numéricas para evitar que la diferencia de magnitudes afecte el cálculo de la distancia.

Una vez preprocesados los datos, el siguiente paso es dividirlos en un conjunto de **entrenamiento** y un conjunto de **prueba**. Esto se realiza utilizando la función train_test_split, que garantiza que el modelo pueda ser evaluado en datos no vistos durante el entrenamiento.

El modelo K-NN se implementa utilizando la clase KNeighborsClassifier de Scikit-Learn. Es posible especificar el número de vecinos (*n_neighbors*) y la métrica de distancia a utilizar (por defecto, la distancia euclideana).

Después de definir el modelo, se ajusta a los datos de entrenamiento mediante el método .fit(). Posteriormente, se realizan las predicciones sobre los datos de prueba con el método .predict().

Para evaluar el desempeño del modelo, se utilizan métricas como la **exactitud (accuracy)**, la **matriz de confusión** y la **puntuación F1**. Estas métricas permiten identificar qué tan precisas son las predicciones y detectar posibles problemas de sobreajuste o subajuste.

Finalmente, es recomendable realizar un análisis visual de los resultados, como la representación gráfica de las fronteras de decisión, especialmente en problemas de clasificación con dos variables numéricas.

Ejemplo práctico y resolución

A continuación, se presenta un ejemplo completo utilizando Scikit-Learn:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Datos
datos = pd.DataFrame({
    'Altura': [180, 165, 170, 160, 175, 155],
    'Peso': [75, 65, 70, 60, 72, 58],
    'Deporte': ['Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto']
})

X = datos[['Altura', 'Peso']]
y = datos['Deporte']
```

```
# Escalamiento
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# División de datos
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.33, random_state=

# Definir y entrenar el modelo
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# Realizar predicciones
y_pred = knn.predict(X_test)
```

```python
# Evaluación
print("Exactitud:", accuracy_score(y_test, y_pred))
print("\nMatriz de Confusión:\n", confusion_matrix(y_test, y_pred))
print("\nReporte de Clasificación:\n", classification_report(y_test, y_pred))
```

Salida posible:

```
Exactitud: 1.0

Matriz de Confusión:
[[1 0]
 [0 1]]

Reporte de Clasificación:
              precision    recall  f1-score   support

  Baloncesto       1.00      1.00      1.00         1
      Fútbol       1.00      1.00      1.00         1

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2
```

Este flujo completo permite implementar y evaluar un clasificador K-NN, garantizando un desempeño adecuado y facilitando la toma de decisiones en tareas de clasificación.

Cierre ✍️

El algoritmo **K-Nearest Neighbors (K-NN)** es una de las técnicas de clasificación más intuitivas y utilizadas en el campo del aprendizaje automático. A través de este manual, hemos recorrido los conceptos fundamentales que sustentan su funcionamiento, como el cálculo de la distancia euclideana, la importancia del escalamiento de datos y la selección del valor óptimo de K.

Hemos visto que, si bien K-NN es un algoritmo fácil de implementar y comprender, también presenta desafíos relacionados con el tiempo de predicción, la escala de las variables y la selección de parámetros. Por ello, su correcta aplicación requiere un preprocesamiento cuidadoso y una evaluación adecuada utilizando métricas como la exactitud y la matriz de confusión.

Además, hemos implementado un flujo completo utilizando la librería **Scikit-Learn**, desde la preparación de los datos hasta la evaluación del modelo. Esto proporciona al lector una base práctica sólida para aplicar este algoritmo en problemas reales de clasificación.

Dominar K-NN no solo permite resolver problemas sencillos, sino que también ayuda a comprender principios fundamentales del aprendizaje automático supervisado, como la proximidad entre datos, la elección de métricas y la validación de modelos.

Referencias📚

- Scikit-learn Developers. (s.f.). *Nearest neighbors — scikit-learn documentation*. https://scikit-learn.org/stable/modules/neighbors.html
- Brownlee, J. (s.f.). *K-Nearest Neighbors for Machine Learning*. Machine Learning Mastery. https://machinelearningmastery.com/k-nearest-neighbors-for-machine-learning/
- Scikit-learn Developers. (s.f.). *Nearest neighbors classification example — scikit-learn documentation*. https://scikit-learn.org/stable/auto_examples/neighbors/plot_classification.html

¡Muchas gracias!

Nos vemos en la próxima lección

alkemy